EHM(X): 形式規則一覧

System F^{ξ} に集合効果 handler と静的述語 X を加える rank-1 表層

Figure 1. 構文・制約・効果の解釈

$$
\begin{aligned}
&e ::= x \mid n \mid \mathbf{true} \mid \mathbf{false} \mid \lambda x.e \mid e_1\, e_2 \mid \mathbf{let}\ x = e_1\ \mathbf{in}\ e_2 \mid \mathbf{perform}\ op\ e \mid \mathbf{handle}\ e\ \mathbf{with}\ h,\\
&h ::= \{\mathbf{return}\ r \mapsto e_r;\ \ op(x;k) \mapsto e_o\}, \qquad v ::= x \mid n \mid \mathbf{true} \mid \mathbf{false} \mid \lambda x.e,\\
&\tau ::= \alpha \mid \mathit{Int} \mid \mathit{Bool} \mid \tau \xrightarrow{\epsilon} \tau, \qquad \epsilon ::= \varnothing \mid \{op\} \mid \mu \mid \epsilon \cup \epsilon,\\
&\sigma ::= \forall \bar{\alpha}, \bar{\mu}.\ P \Rightarrow \tau,\\
&C ::= \top \mid C \wedge C \mid \tau = \tau \mid \epsilon \subseteq \epsilon \mid op \notin \epsilon \mid p_X \mid \forall \alpha.C \mid \forall \mu.C \mid \exists \alpha.C \mid \exists \mu.C.
\end{aligned}
$$

$$
\epsilon \equiv \epsilon' \quad \overset{\text{def}}{\iff} \quad (\epsilon \subseteq \epsilon') \wedge (\epsilon' \subseteq \epsilon).
$$

$$
\mathrm{NF}(\epsilon) = (L_\epsilon, V_\epsilon), \qquad I_\rho(\epsilon) = L_\epsilon \cup \bigcup_{\mu \in V_\epsilon} \rho(\mu), \qquad I_\rho(\{\mathit{Read}\} \cup \mu) = \{\mathit{Read}\} \cup \rho(\mu).
$$

$$
I_\rho(\alpha) = \rho_T(\alpha), \quad I_\rho(\mathit{Int}) = \mathit{Int}, \quad I_\rho(\mathit{Bool}) = \mathit{Bool}, \quad I_\rho(a \xrightarrow{\epsilon} b) = I_\rho(a) \xrightarrow{I_\rho(\epsilon)} I_\rho(b).
$$

$$
\boxed{\mathrm{Split}(op, \epsilon, \delta) \overset{\text{def}}{=} (\delta \subseteq \epsilon) \wedge (\epsilon \subseteq \{op\} \cup \delta) \wedge (op \notin \delta)}
$$

$$
\rho \models \mathrm{Split}(op, \epsilon, \delta) \quad \iff \quad I_\rho(\delta) = I_\rho(\epsilon) \setminus \{op\}.
$$

Figure 2. 値呼びの簡約規則

$$
(\lambda x.e)\, v \longrightarrow e[x \mapsto v] \qquad \mathbf{handle}\ v\ \mathbf{with}\ h \longrightarrow e_r[r \mapsto v].
$$

$$
\mathbf{handle}\ E[\mathbf{perform}\ op\ v]\ \mathbf{with}\ h \longrightarrow e_o[x \mapsto v][k \mapsto k_E],
$$
$$
op \notin \mathrm{handled}(E), \qquad k_E = \lambda y.\mathbf{handle}\ E[y]\ \mathbf{with}\ h.
$$

宣言的型付け　$C; \Gamma \vdash e : \tau \mid \epsilon$

Figure 3. 基本規則

$$\frac{\Gamma(x) = \sigma = \forall \bar{\beta}.P \Rightarrow \tau \quad \mathrm{SafeRename}(\sigma, \theta) \quad C \models P[\bar{\theta}/\bar{\beta}]}{C; \Gamma \vdash x : \tau[\bar{\theta}/\bar{\beta}] \mid \varnothing} \quad (\textsc{Var})$$

$$\frac{C; \Gamma, x : a \vdash e : b \mid \eta}{C; \Gamma \vdash \lambda x.e : a \xrightarrow{\eta} b \mid \varnothing} \quad (\textsc{Abs})$$

$$\frac{C; \Gamma \vdash e_1 : a \xrightarrow{\eta} b \mid \epsilon_1 \quad C; \Gamma \vdash e_2 : a \mid \epsilon_2}{C; \Gamma \vdash e_1\, e_2 : b \mid \epsilon_1 \cup \epsilon_2 \cup \eta} \quad (\textsc{App})$$

$$\frac{\Sigma(op) = \forall \bar{\beta}.\tau_a \to \tau_b \quad C; \Gamma \vdash e : \tau_a[\bar{\theta}/\bar{\beta}] \mid \eta}{C; \Gamma \vdash \mathbf{perform}\ op\ e : \tau_b[\bar{\theta}/\bar{\beta}] \mid \eta \cup \{op\}} \quad (\textsc{Perform})$$

$$\frac{n \in \mathbb{Z}}{C; \Gamma \vdash n : Int \mid \varnothing} \quad (\textsc{Int}) \qquad \frac{b \in \{\mathbf{true}, \mathbf{false}\}}{C; \Gamma \vdash b : Bool \mid \varnothing} \quad (\textsc{Bool})$$

$\mathrm{SafeRename}(\sigma, \theta)$:　$|\theta B| = |B|$,　　$\theta B \cap (\mathrm{FV}(\sigma) \cup \mathrm{Bound}(P)) = \varnothing$　（型・効果の各変数ごと）.

Figure 4. 値制限付き let

$$\frac{C_1; \Gamma \vdash v : \tau_1 \mid \varnothing \quad A = \mathrm{FV}(C_1, \tau_1) \setminus \mathrm{FV}(\Gamma) \quad \sigma = \forall A.C_1 \Rightarrow \tau_1, \quad R_1 = \exists A.C_1, \quad C_2; \Gamma, x : \sigma \vdash e_2 : \tau_2 \mid \epsilon_2}{R_1 \wedge C_2; \Gamma \vdash \mathbf{let}\ x = v\ \mathbf{in}\ e_2 : \tau_2 \mid \epsilon_2} \quad (\textsc{LetVal})$$

$$\frac{\neg\, \mathrm{Value}(e_1) \quad C; \Gamma \vdash e_1 : \tau_1 \mid \epsilon_1 \quad C; \Gamma, x : \tau_1 \vdash e_2 : \tau_2 \mid \epsilon_2}{C; \Gamma \vdash \mathbf{let}\ x = e_1\ \mathbf{in}\ e_2 : \tau_2 \mid \epsilon_1 \cup \epsilon_2} \quad (\textsc{LetMono})$$

Figure 5. Deep handler

$$\frac{\begin{array}{c}\Sigma(op) = \forall \overline{\beta :: \kappa}.\tau_a \to \tau_b \qquad C; \Gamma \vdash e : \tau_{in} \mid \epsilon_{in} \\ C; \Gamma, r : \tau_{in} \vdash e_r : \tau_{out} \mid \epsilon_r \\ D; \Gamma, x : \tau_a, k : \tau_b \xrightarrow{\chi} \tau_{out} \vdash e_o : \tau_o \mid \epsilon_o \\ C \models \forall \bar{\beta}.\exists U_T, U_E.(D \wedge \tau_o = \tau_{out} \wedge \epsilon_o \subseteq \chi) \\ C \models \mathrm{Split}(op, \epsilon_{in}, \delta) \quad C \models \delta \subseteq \chi \quad C \models \epsilon_r \subseteq \chi \\ \mathrm{ClauseScope}(C, \Gamma, \tau_{in}, \tau_{out}, \epsilon_{in}, \epsilon_r, \delta, \chi, U_T, U_E)\end{array}}{C; \Gamma \vdash \mathbf{handle}\ e\ \mathbf{with}\ \{\mathbf{return}\ r \mapsto e_r;\ \ op(x; k) \mapsto e_o\} : \tau_{out} \mid \chi} \quad (\textsc{Handle})$$

$\bar{\beta}$ universally checked in the operation clause,　　$\bar{\beta} \cap \mathrm{FV}(\Gamma, \tau_{in}, \tau_{out}, \epsilon_{in}, \epsilon_r, \chi, \delta) = \varnothing$.

$$U_T \cap \mathrm{FV}_T(\Gamma, C, \tau_{in}, \tau_{out}, \tau_a, \tau_b, \bar{\beta}) = \varnothing,$$
$$U_E \cap \mathrm{FV}_E(\Gamma, C, \tau_{in}, \tau_{out}, \epsilon_{in}, \epsilon_r, \delta, \chi, \tau_a, \tau_b, \bar{\beta}) = \varnothing,$$
$$\bar{\beta} \cap \mathrm{FV}(C) = \varnothing.$$

操作宣言の束縛名は，必要なら節に入る前に捕獲を避けて α 改名する.

$$d' = \theta d, \quad B'_T = \theta_T B_T, \quad B'_E = \theta_E B_E,$$
$$|B'_T| = |B_T|, \quad |B'_E| = |B_E|, \quad B'_T \cap (\mathrm{FV}_T(\tau_a, \tau_b) \setminus B_T) = \varnothing,$$
$$B'_E \cap (\mathrm{FV}_E(\tau_a, \tau_b) \setminus B_E) = \varnothing.$$

制約生成　$\mathrm{Gen}_\Gamma(e) = (\tau, \epsilon, C)$

Figure 6. 構文に沿った生成方程式

$$\mathrm{Gen}_\Gamma(n) = (\mathit{Int}, \varnothing, \top), \qquad \mathrm{Gen}_\Gamma(b) = (\mathit{Bool}, \varnothing, \top) \quad (b \in \{\mathbf{true}, \mathbf{false}\}).$$

$$\Gamma(x) = \sigma = \forall \bar{\beta}.P \Rightarrow \tau_x, \quad \theta \text{ は } \bar{\beta} \text{ の許容される改名} \Longrightarrow \mathrm{Gen}_\Gamma(x) = (\theta\tau_x, \varnothing, \theta P),$$
$$\mathrm{Gen}_{\Gamma, x:a}(e) = (b, \eta, C) \Longrightarrow \mathrm{Gen}_\Gamma(\lambda x.e) = (a \xrightarrow{\eta} b, \varnothing, C).$$

$$\mathrm{Gen}_\Gamma(e_1) = (\tau_1, \epsilon_1, C_1), \quad \mathrm{Gen}_\Gamma(e_2) = (\tau_2, \epsilon_2, C_2) \Longrightarrow \mathrm{Gen}_\Gamma(e_1\, e_2)$$
$$= (b, \epsilon_1 \cup \epsilon_2 \cup \eta, C_1 \wedge C_2 \wedge (\tau_1 = \tau_2 \xrightarrow{\eta} b)).$$

$$\Sigma(\mathit{op}) = \forall \bar{\beta}.\tau_a \to \tau_b, \quad \mathrm{Gen}_\Gamma(e) = (\tau, \eta, C) \Longrightarrow \mathrm{Gen}_\Gamma(\mathbf{perform}\ \mathit{op}\ e)$$
$$= (\theta\tau_b, \{\mathit{op}\} \cup \eta, C \wedge (\tau = \theta\tau_a)).$$

a, b, η, θ は使用時に新たに選ぶ。

Figure 7. let の生成と局所的な一般化

$$\mathrm{Gen}(\Gamma, v) = (\tau_1, \varnothing, C_1), \qquad A = \mathrm{FV}(\tau_1, C_1) \setminus \mathrm{FV}(\Gamma),$$
$$\sigma = \forall A.C_1 \Rightarrow \tau_1, \qquad R_1 = \exists A.C_1,$$
$$\mathrm{Gen}_{\Gamma, x:\sigma}(e_2) = (\tau_2, \epsilon_2, C_2).$$

$$\mathrm{Gen}_\Gamma(\mathbf{let}\ x = v\ \mathbf{in}\ e_2) = (\tau_2, \epsilon_2, R_1 \wedge C_2),$$

$$\mathrm{Gen}_\Gamma(e_1) = (\tau_1, \epsilon_1, C_1), \quad \mathrm{Gen}_{\Gamma, x:\tau_1}(e_2) = (\tau_2, \epsilon_2, C_2) \Longrightarrow \mathrm{Gen}_\Gamma(\mathbf{let}\ x = e_1\ \mathbf{in}\ e_2)$$
$$= (\tau_2, \epsilon_1 \cup \epsilon_2, C_1 \wedge C_2), \quad \neg\,\mathrm{Value}(e_1).$$

$$\rho \models R_1 \quad \Longleftrightarrow \quad \exists \rho'.\ \rho' \models C_1 \ \wedge\ \rho'|_{\overline{A}} = \rho|_{\overline{A}}, \qquad \mathrm{FV}(R_1) \subseteq \mathrm{FV}(\Gamma).$$

Handler の生成・解決・健全性

Figure 8. Handler の制約生成

$$\Sigma(op) = \forall\overline{\beta :: \kappa}.\tau_a \to \tau_b, \qquad a, \delta, \chi \text{ fresh},$$
$$\mathrm{Gen}_\Gamma(e) = (\tau_{in}, \epsilon_{in}, C_{in}),$$
$$\mathrm{Gen}_{\Gamma, r:\tau_{in}}(e_r) = (\tau_r, \epsilon_r, C_r),$$
$$\mathrm{Gen}_{\Gamma, x:\tau_a, k:\tau_b \xrightarrow{\chi} a}(e_o) = (\tau_o, \epsilon_o, C_o).$$

$U_T, U_E =$ fresh type and effect names allocated during $\mathrm{Gen}(e_o)$.

$$\begin{aligned}\mathrm{Gen}_\Gamma(\textbf{handle } e \textbf{ with } h) = (a, \chi, C),\\ C = C_{in} \wedge C_r \wedge (\tau_r = a)\\ \wedge\, \forall\bar{\beta}.\exists U_T, U_E.(C_o \wedge \tau_o = a \wedge \epsilon_o \subseteq \chi)\\ \wedge\, \mathrm{Split}(op, \epsilon_{in}, \delta) \wedge \delta \subseteq \chi \wedge \epsilon_r \subseteq \chi.\end{aligned}$$

$\bar{\beta}$ rigid; $\qquad \bar{\beta} \cap \mathrm{FV}(\Gamma, \tau_{in}, a, \delta, \chi, \epsilon_{in}, \epsilon_r) = \varnothing.$

$$\mathrm{ClauseScope}(C, \Gamma, \tau_{in}, a, \epsilon_{in}, \epsilon_r, \delta, \chi, U_T, U_E).$$

Figure 9. 解決器の保存条件

$$(a_1 \xrightarrow{\eta_1} b_1) = (a_2 \xrightarrow{\eta_2} b_2) \rightsquigarrow (a_1 = a_2) \wedge (b_1 = b_2) \wedge (\eta_1 \equiv \eta_2),$$
$$\mathrm{simplify}(C) = \mathrm{some}(R) \implies \forall\rho.\ \rho \models R \iff \rho \models C,$$
$$\mathrm{resolve}_X(A, C) = R_X \implies \forall\rho.\ \rho \models A \wedge R_X \iff \rho \models A \wedge C.$$

これは解集合の保存であり，生成器全体の完全性を含まない。

Figure 10. *ping* **の制約簡約**

$$\Sigma(ping) = \forall\beta.\beta \to \beta, \qquad f : \forall\alpha.\ Eq\,\alpha \Rightarrow \alpha \xrightarrow{\varnothing} \alpha,$$
$$g = \lambda x.\textbf{perform } ping\,(f\,x), \qquad g : \forall\alpha.\ Eq\,\alpha \Rightarrow \alpha \xrightarrow{\{ping\}} \alpha,$$
$$e = \textbf{handle } (g\,y) \textbf{ with } \{\textbf{return } r \mapsto r;\ ping(x; k) \mapsto k\,x\},$$
$$\mathrm{Split}(ping, \{ping\}, \delta) \implies \delta = \varnothing, \qquad \epsilon_r = \varnothing, \qquad \epsilon_o = \chi,$$
$$C_e = Eq\,\alpha \wedge (\varnothing \subseteq \chi) \wedge (\chi \subseteq \chi), \qquad \chi = \varnothing \text{ is one solution.}$$

Figure 11. 完全性・制約付き主要型の主張範囲

$$\mathrm{Sol}(C) = \{\rho \mid \rho \models C\},$$
$$\mathrm{simplify}(C) = \mathrm{some}(R) \Longrightarrow \mathrm{Sol}(C) = \mathrm{Sol}(R),$$
$$\mathrm{Sol}(\mathrm{Assume}(A) \wedge C) = \mathrm{Sol}(\mathrm{Assume}(A) \wedge \mathrm{resolve}_X(A, C)).$$

$$\mathrm{Inst}_\rho(\forall A.P \Rightarrow \tau) = \{I_{\rho'}(\tau) \mid \rho' \models P,\ \rho'|_{\overline{A}} = \rho|_{\overline{A}}\}, \qquad \sigma_g \preceq_\rho \sigma_d \Longleftrightarrow \mathrm{Inst}_\rho(\sigma_d) \subseteq \mathrm{Inst}_\rho(\sigma_g).$$

$$\mathrm{Closed}(\Sigma) \Longleftrightarrow \forall\, \Sigma(op) = \forall B_T, B_E.\tau_a \to \tau_b. \quad \mathrm{FV}_T(\tau_a, \tau_b) \subseteq B_T \ \wedge\ \mathrm{FV}_E(\tau_a, \tau_b) \subseteq B_E.$$

$$\mathrm{Used}_T(C) = \mathrm{FV}_T(C) \cup \mathrm{BV}_T(C), \quad \mathrm{Supp}_T(r) = \mathrm{FV}_T(r.\tau) \cup \mathrm{Used}_T(r.C),$$
$$\mathrm{Used}_E(C) = \mathrm{FV}_E(C) \cup \mathrm{BV}_E(C), \quad \mathrm{Supp}_E(r) = \mathrm{FV}_E(r.\tau) \cup \mathrm{Vars}(r.\epsilon) \cup \mathrm{Used}_E(r.C).$$

$$\left.\begin{array}{c}\mathrm{Supp}_T(r) \subseteq [0, r.next_T), \quad \mathrm{Supp}_E(r) \subseteq [0, r.next_E),\\ \rho|_{[0,r.next)} = \rho'|_{[0,r.next)}\end{array}\right\} \Longrightarrow \begin{array}{c}\rho \models r.C \Longleftrightarrow \rho' \models r.C,\\ I_\rho(r.\tau) = I_{\rho'}(r.\tau), \quad I_\rho(r.\epsilon) = I_{\rho'}(r.\epsilon).\end{array}$$

$$\boxed{\mathrm{Closed}(\Sigma) \wedge \mathrm{LawfulRename}(X) \wedge \mathrm{LawfulSupport}(X) \wedge \mathrm{LawfulFVRename}(X)}$$

$$\boxed{\text{handler を含む全項の生成完全性・推論結果の主要性を Lean で証明済み}}$$

必要な補題・仮定	現状
X の改名・有限支持・代入法則	Eq/Show について Lean で証明済み．改名後の自由名は元の自由名の像に含まれる
新変数と操作宣言の捕獲回避	seed の上界と再帰中の単調性を Lean で証明済み．変数参照時の seed の安全性も証明済み
scheme 具体例と有限範囲の改名	双方向の意味保存，既存の名前を保つ具体例の再現を Lean で証明済み
値 **let** の $\exists A.C_1$	Lean・Rust で実装，自由変数の範囲を Lean で証明済み
閉じた操作シグネチャ	Closed(Σ) を Lean に定義し，操作宣言の自由名がない場合の `perform` の逆向きを証明済み
操作節の $\forall\bar{\beta}.\exists U_T, U_E$	閉じた宣言の改名逆対応，文脈の具体例包含，節の局所射影を Lean で証明済み
制約簡約の解集合の保存	Lean で証明済み
基本部分の宣言的型付けから生成規則への逆向き	Lean で証明済み
値 let の scheme 具体例の包含	Lean で証明済み
異なる文脈での値 let の結合	scheme の包含，残余制約，本体への結合を Lean で証明済み
文脈の scheme 包含と変数ケース	公開入口 `generateFresh` の変数ケースまで Lean で証明済み
後続の部分式で先行制約を保つ補題	支持集合の意味保存と，handler を含む生成器全体の供給上界を Lean で証明済み
handler の scope 判定	生成した body・return・操作節から `HandlerScope` と `ClauseScope` の成立を Lean で証明済み
文脈全体・handler を含む完全性と主要型	`Typing.generate_complete`, `Typing.generateFresh_principal`, `Typing.inferQualifiedTotal_principal` を Lean で証明済み

1 主要な補題・命題・定理

$$R \models C \ \overset{\mathrm{def}}{\Longleftrightarrow}\ \forall \rho.\ \rho \models R \Rightarrow \rho \models C, \qquad \mathrm{Assume}(A) = \bigwedge_{p \in A} p.$$

補題 1.1 (操作の厳密な除去)**.** 任意の評価 ρ について,

$$\rho \models \mathrm{Split}(op, \epsilon, \delta) \quad \Longleftrightarrow \quad I_\rho(\delta) = I_\rho(\epsilon) \setminus \{op\}.$$

補題 1.2 (制約の有限支持)**.** X の述語が自由変数の外側の評価に依存しないとする．ρ, ρ' が $\mathrm{FV}(C)$ 上で一致すれば,

$$\rho \models C \quad \Longleftrightarrow \quad \rho' \models C.$$

この補題は $\forall$ と $\exists$ の両方を含む制約について成り立つ.

補題 1.3 (束縛名の捕獲回避)**.**

$$\mathrm{renameBound}(B, b)(i) = \begin{cases} b + i + 1 & i \in B, \\ i & i \notin B. \end{cases}$$

供給の更新は $i \in B$ に対して $\mathrm{renameBound}(B, b)(i) < b + \sup B + 2$ を満たす．使用中の自由名と内部束縛名がすべて b 以下なら，*scheme* の SafeRename が成り立つ．操作宣言でも同じ条件から AlphaSafe が成り立つ.

補題 1.4 (scheme の安全な具体化)**.** 改名 ν_T, ν_E が *scheme* の束縛名を有限範囲で衝突なく写し，自由名と制約内部の束縛名を捕獲しないとする．このとき

$$\rho \models P[\nu_T, \nu_E] \Longrightarrow I_\rho(\tau[\nu_T, \nu_E]) \in \mathrm{Inst}_\rho(\forall A.P \Rightarrow \tau).$$

逆に $t \in \mathrm{Inst}_\rho(\sigma)$ なら，改名後の束縛名に値を割り当てた評価 ρ' が存在して

$$\rho' \models P[\nu_T, \nu_E], \qquad I_{\rho'}(\tau[\nu_T, \nu_E]) = t.$$

改名先が固定集合 F_T, F_E と交わらなければ,

$$\rho'|_{F_T, F_E} = \rho|_{F_T, F_E}.$$

Lean: ***Scheme.instantiate_groundInstance, Scheme.groundInstance_realizable_fixed.***

命題 1.5 (変数参照の包含)**.** $\Gamma_g \preceq \Gamma_d$ とし，宣言側の変数参照が t を持つとする．生成側で安全に改名すれば，既存の名前を保つ評価 ρ' が存在して

$$\rho' \models P_g[\nu_T, \nu_E], \qquad I_{\rho'}(\tau_g[\nu_T, \nu_E]) = t.$$

公開入口の *seed* は文脈内の *scheme* の使用名より大きいため，変数参照の安全性判定は成功する．さらに宣言側の任意の具体例を生成結果の解として再現できる．*Lean* では ***Context.variable_complete_at*** と ***generateFresh_var_ground_complete.***

補題 1.6 (実行可能な生成器の名前上界)**.**

$$\mathrm{generateFresh}(\Sigma, X, \Gamma, e, s) = \mathrm{some}(r) \Longrightarrow \mathrm{Supp}_T(r) \subseteq [0, r.next_T) \ \wedge \ \mathrm{Supp}_E(r) \subseteq [0, r.next_E).$$

ここで制約内部の量化子名も Supp に含める．この上界と有限支持から，後続の再帰呼び出しが $r.next$ 以上の名前だけを変更する場合，$r.C$ の充足と $r.\tau, r.\epsilon$ の評価が保存される．*Lean:* ***generate_support_below, generateFresh_support_below, generateFresh_solution_frame.***

補題 1.7 (局所変数の射影)**.** 型変数集合 U_T と効果変数集合 U_E について,

$$\rho \models \exists U_T, U_E.C \Longleftrightarrow \exists \rho'.\ \rho' \models C \ \wedge \ \rho'|_{\overline{U_T}, \overline{U_E}} = \rho|_{\overline{U_T}, \overline{U_E}}.$$

特に $C \models \exists U_T, U_E.C$ である.

補題 1.8 (値 let の残余制約)**.** $A = \mathrm{FV}(C_1, \tau_1) \setminus \mathrm{FV}(\Gamma)$ とする．$R_1 = \exists A.C_1$ について,

$$C_1 \models R_1, \qquad \rho \models R_1 \Longleftrightarrow \exists \rho'.\ \rho' \models C_1 \wedge \rho'|_{\overline{A}} = \rho|_{\overline{A}}, \qquad \mathrm{FV}(R_1) \subseteq \mathrm{FV}(\Gamma).$$

補題 1.9 (値 let の scheme 具体例の包含)**.** $C_1 \models C_g$ かつ C_1 の各解で $I(\tau_g) = I(\tau_1)$ とする．X の述語が有限支持を満たせば,

$$\mathrm{Inst}_\rho(\mathrm{gen}(\Gamma, C_1, \tau_1)) \subseteq \mathrm{Inst}_\rho(\mathrm{gen}(\Gamma, C_g, \tau_g)).$$

Lean では ***Scheme.generalize_moreGeneralAt.***

補題 1.10 (異なる文脈での値 let の結合)**.** $\Gamma_g \preceq_{\rho_g,\rho_d} \Gamma_d$ とし，宣言側の値制約 C_1 の各解で，生成側に対応する解が存在し，Γ_g の自由名を保ち，結果型・効果の評価が等しいとする．このとき

$$\mathrm{gen}(\Gamma_g, C_g, \tau_g) \preceq_{\rho_g,\rho_d} \mathrm{gen}(\Gamma_d, C_1, \tau_1), \qquad \rho_d \models \exists A_d.C_1 \Longrightarrow \rho_g \models \exists A_g.C_g.$$

本体の制約生成が拡張後の文脈で同じ対応を満たせば，値 *let* 全体の生成制約も対応する．X には有限支持だけを仮定する．*Lean* では *`Scheme.generalize_moreGeneralAt_across`*, *`Scheme.residual_across`*, *`GeneratedCoversAcross.letVal`*.

命題 1.11 (基本部分の生成完全性)**.** e に値 *let* と *handler* が現れず，$C;\Gamma \vdash e : \tau \mid \epsilon$ なら，ある生成結果 $(\tau_g, \epsilon_g, C_g)$ が存在して，

$$\forall \rho.\ \rho \models C \Longrightarrow \rho \models C_g \ \wedge\ I_\rho(\tau_g) = I_\rho(\tau) \ \wedge\ I_\rho(\epsilon_g) = I_\rho(\epsilon).$$

Lean では *`Typing.core_generate_complete`*.

定理 1.12 (EHM(X) の生成完全性と推論結果の主要性)**.** Closed(Σ) と LawfulRename(X), LawfulSupport(X), LawfulFVRename(X) を仮定する．*handler* を含め，$C;\Gamma \vdash e : \tau \mid \epsilon$ なら，実行可能な生成器は

$$\mathrm{generateFresh}(\Sigma, X, \Gamma, e, s) = \mathrm{some}(r)$$

を満たす．生成結果そのものが $r.C;\Gamma \vdash e : r.\tau \mid r.\epsilon$ を満たす．帰納法で示す強い形は，Γ_g の *scheme* が Γ_d より一般的で，供給 s が入力名より大きいとき，

$$\begin{aligned}\Gamma_g \preceq_{\rho_g,\rho_d} \Gamma_d \ \wedge\ \rho_d \models C \Longrightarrow \exists w.\quad & w|_{[0,s)} = \rho_g|_{[0,s)} \ \wedge\ w \models r.C \\ & \wedge I_w(r.\tau) = I_{\rho_d}(\tau) \ \wedge I_w(r.\epsilon) = I_{\rho_d}(\epsilon),\end{aligned}$$

ただし $\mathrm{generate}(\Sigma, X, \Gamma_g, e, s) = \mathrm{some}(r)$．制約付き型 $Q = (C, \tau, \epsilon)$ の具体例関係を

$$\begin{aligned}Q_g \succeq_\Gamma Q_d \iff \forall \rho_d.\ \rho_d \models C_d \Longrightarrow \exists \rho_g.\quad & \rho_g|_{\mathrm{FV}(\Gamma)} = \rho_d|_{\mathrm{FV}(\Gamma)} \\ & \wedge \rho_g \models C_g \ \wedge\ I_{\rho_g}(\tau_g) = I_{\rho_d}(\tau_d) \ \wedge\ I_{\rho_g}(\epsilon_g) = I_{\rho_d}(\epsilon_d)\end{aligned}$$

と定義する．型変数と効果変数の両方について文脈の自由名を固定する．これは *HM(X)* の *constrained type* の具体例順序を，型と効果の対に拡張したものである．生成結果 r 自身に型がつき，任意の宣言的導出 $C';\Gamma \vdash e : \tau' \mid \epsilon'$ に対して

$$\boxed{(r.C, r.\tau, r.\epsilon) \succeq_\Gamma (C', \tau', \epsilon')}$$

が成り立つ．さらに明示的仮定 A 付きの実行可能な推論結果 $q = \mathrm{inferQualifiedTotal}(\Sigma, X, A, \Gamma, e, s)$ について

$$\boxed{(q.C, q.\tau, q.\epsilon) \succeq_\Gamma (\mathrm{Assume}(A) \wedge C', \tau', \epsilon')}$$

が成り立つ．$\mathrm{Assume}(A) \wedge C'$ に解があれば $q.C$ にも解がある．*Lean* で対応する定理を証明済み：
`Typing.generate_complete`, *`Typing.generateFresh_principal`*,
`Typing.inferQualifiedTotal_principal`,
`Typing.inferQualifiedTotal_realizable`.

主要性はこの具体例順序に関する主張である．制約を正規形まで解く主張には，*HM(X)* と同様，X の *principal normal form* 条件が別途必要である．

補題 1.13 (量化した操作節での包含保存)**.** D の各解で C_o が成り立ち，節の生成結果型・効果が宣言側と等しいとする．$Q_d = D \wedge (\tau_o = a) \wedge (\epsilon_o \subseteq \chi)$，$Q_g = C_o \wedge (\tau_{og} = a) \wedge (\epsilon_{og} \subseteq \chi)$ と置けば，

$$\mathrm{Close}_B\big(\exists U.Q_d\big) \models \mathrm{Close}_B\big(\exists U.Q_g\big).$$

Lean では *`Constraint.closed_clause_mono`*.

補題 1.14 (操作節の量化順序)**.**

$$\begin{aligned}\rho \models \forall B_T, B_E.\exists U_T, U_E.C \iff \forall \rho_B \equiv_{\overline{B}} \rho.\\ \exists \rho_U \equiv_{\overline{U}} \rho_B.\ \rho_U \models C.\end{aligned}$$

操作の剛性変数を任意に決めた後で，節の局所変数を選ぶ．

補題 1.15 (操作節の局所解の射影)**.** 節の生成結果を r，開始供給を s とし，$U_T = [s_T, r.next_T)$，$U_E = [s_E, r.next_E)$ と置く．宣言的な節の解に対応する生成解 w があり，w が開始供給より小さい名前を保存し，r の支持が $r.next$ 未満なら，ある w' が存在して

$$w'|_{\overline{U}} = \rho|_{\overline{U}}, \qquad w' \models r.C, \qquad I_{w'}(r.\tau) = I_w(r.\tau), \quad I_{w'}(r.\epsilon) = I_w(r.\epsilon).$$

したがって，節の結果型と効果の制約も保たれる場合，生成された節の存在量化制約を満たす．*Lean* では *`Output.localize_solution`* と *`Output.CoversAt.project_clause`*.

補題 1.16 (handler の外側の名前分離)**.** 生成器が e を開始供給 s から生成して r を返した場合，

$$\mathrm{Vars}(r.\epsilon) \subseteq [s_E, \infty).$$

これと *body* の支持上界から，*return* 節の効果は新たに束縛した操作変数を参照しない．生成した *body*・*return* に対して HandlerScope が成立する．*Lean* では *`generate_effect_fresh`* と *`generated_handlerScope_of_runs`*.

補題 1.17 (操作節の名前分離と逆向きの具体化)**.** Closed(Σ) の下で，生成した *return* 節が開始供給より小さい名前を使用するなら，その名前は *body* の結果または外側の文脈に由来する．したがって操作の束縛名は *return* 節と交わらず，操作節の局所名は外側の自由名と交わらない．生成器の ClauseScope 判定は成功する．任意の生成側の操作束縛名の評価には，宣言側の束縛名の評価を対応付けられる．*Lean:* *`generate_support_origin`*, *`generated_return_old_support`*, *`generated_clauseScope_of_bounds`*, *`OpDecl.closed_rename_reverse`*.

補題 1.18 (操作節の局所的な具体化)**.** $i \neq j$ ならば，任意の評価 ρ について

$$\rho \models \forall\alpha_i.\exists\alpha_j.(\alpha_j = \alpha_i).$$

一方，i, j, k が互いに異なれば

$$\neg\exists\rho.\ \rho \models \forall\alpha_i.\exists\alpha_k.(\alpha_i = \alpha_j).$$

補題 1.19 (剛性変数の全称閉包)**.** 型変数・効果変数の有限集合 B_T, B_E について，

$$\mathrm{Close}_{B_T,B_E}(D) \stackrel{\mathrm{def}}{=} \forall B_T.\forall B_E.D, \qquad \mathrm{Close}_{B_T,B_E}(D) \models D.$$

命題 1.20 (多相操作引数の固定結果型への漏出)**.** $i \neq j$ ならば，

$$\neg\exists\rho.\ \rho \models \forall\alpha_i.(\alpha_i = \alpha_j).$$

補題 1.21 (値の生成効果)**.**

$$\mathrm{Value}(v) \ \wedge\ \mathrm{Gen}_\Gamma(v) = (\tau, \epsilon, C) \quad \Longrightarrow \quad \epsilon = \varnothing.$$

命題 1.22 (宣言的型付けに対する生成規則の健全性)**.**

$$\mathrm{Gen}_\Gamma(e) = (\tau, \epsilon, C) \quad \Longrightarrow \quad C; \Gamma \vdash_{\mathrm{EHM}(X)} e : \tau \mid \epsilon.$$

命題 1.23 (実行可能な生成器の規則への適合)**.**

$$\mathrm{generate}(\Sigma, X, \Gamma, e, s) = \mathrm{some}(r) \quad \Longrightarrow \quad \mathrm{Gen}_\Gamma(e) = (r.\tau, r.\epsilon, r.C).$$

補題 1.24 (簡約器の健全性)**.**

$$\mathrm{simplify}(C) = \mathrm{some}(R) \quad \Longrightarrow \quad R \models C \ \wedge\ C \models R.$$
$$\mathrm{simplify}(C) = \mathrm{none} \quad \Longrightarrow \quad \neg\exists\rho.\ \rho \models C.$$

補題 1.25 (追加述語の解決の同値性)**.**

$$\mathrm{Assume}(A) \wedge \mathrm{resolve}_X(A, C) \ \equiv\ \mathrm{Assume}(A) \wedge C.$$

定理 1.26 (EHM(X) 推論の健全性)**.**

$$\mathrm{inferQualified}(\Sigma, X, A, \Gamma, e, s) = \mathrm{some}(r) \quad \Longrightarrow \quad r.C; \Gamma \vdash_{\mathrm{EHM}(X)} e : r.\tau \mid r.\epsilon.$$

ここで $r.C$ は $\mathrm{Assume}(A)$ と未解決の制約を含む.

命題 1.27 (生成後の解集合の保存)**.** $\mathrm{inferQualified}(\Sigma, X, A, \Gamma, e, s) = \mathrm{some}(r)$ ならば，ある生成結果 g が存在し，

$$\mathrm{generate}(\Sigma, X, \Gamma, e, s) = \mathrm{some}(g), \quad r.\tau = g.\tau, \quad r.\epsilon = g.\epsilon,$$
$$\forall \rho. \quad \rho \models r.C \iff \rho \models \mathrm{Assume}(A) \wedge g.C.$$

仮定の必要性を示す反例. X_{bad} で p_0 は真, p_1 は偽とし, $\rho_T(0) \neq 0$ の改名が p_0 を p_1 に変えるとする. $\Gamma = [\forall \alpha_0. p_0 \Rightarrow \alpha_0]$ では $\top; \Gamma \vdash x : \alpha_0 \mid \varnothing$ だが, 新変数への改名を行う生成器は充足不能な p_1 を出す. これは $\mathrm{LawfulRename}(X)$ に違反するため, 同条件を置いた完全性の反例ではない. 供給変数の衝突も Fresh により除外する. Lean: `classDomain_lawfulRename`, `classDomain_lawfulSupport`, `nonLawfulDomain_declarative_var`, `nonLawfulDomain_generated_unsat`.

$ping(x; k) \mapsto x$ は上の命題により充足不能となる．$ping(x; k) \mapsto k\, x$ では $\chi = \varnothing$ は一解だが，この推論器は最小効果を選ばず χ と包含制約を残す.

対応する Lean 定理：`Constraint.split_iff`, `Constraint.holds_support`, `Constraint.project_holds_iff`, `Scheme.residual_holds_iff`, `Scheme.residual_fv_context`.
`Constraint.close_project_holds_iff`, `Constraint.close_current`, `Constraint.rigid_var_eq_unsat`, `local_instance_after_rigid`, `projected_argument_cannot_escape`.
`Generate.value_effect`, `Generate.sound`, `generate_sound`.
`simplify_equiv`, `resolveX_equiv`, `simplify_none_unsat`, `inferQualified_sound`, `inferQualified_preserves_solutions`.

2 操作節の局所変数の射影

$$\Sigma(ping) = \forall \beta. \beta \to \beta, \qquad id : \forall \gamma. \gamma \xrightarrow{\varnothing} \gamma,$$
$$H_{id} = \lambda y. \mathbf{handle}\ (\mathbf{perform}\ ping\ y)\ \mathbf{with}\ \{\mathbf{return}\ r \mapsto r;\ ping(x; k) \mapsto k\,(id\ x)\}.$$

操作節で id を具体化するときに生成する u は，各 β に依存してよい．主要な制約は次の形になる.

$$\forall \beta. \exists u. \big(u = \beta\ \wedge\ u \xrightarrow{\varnothing} u = \beta \xrightarrow{\varnothing} \beta \wedge\ a = a\big).$$

$u := \beta$ を選べるので，この部分は充足可能である．旧生成器の $\exists u. \forall \beta. (u = \beta)$ は充足不能だった．ここで射影するのは操作節の生成中に新たに確保した名前だけである．handler の結果型 a や操作の剛性変数 β は射影しない.

$ping(x; k) \mapsto x$ では節に新しい名前がない．従って $\forall \beta. \exists \varnothing. (\beta = a)$ は $\forall \beta. (\beta = a)$ と同じであり，依然として充足不能である.

3　導出木：$ping(x;k) \mapsto x$ の制約

$$\Sigma(ping) = \forall\alpha.\alpha \to \alpha, \qquad \sigma_{id} = \forall\gamma.\gamma \xrightarrow{\varnothing} \gamma, \qquad T = \theta \xrightarrow{\varnothing} \theta,$$
$$B = \textbf{let } id = \lambda z.z \textbf{ in perform } ping\ id,$$
$$H = \textbf{handle } B \textbf{ with } \{\textbf{return } r \mapsto r;\ ping(x;k) \mapsto x\}.$$

本体の導出． id は σ_{id} から T に具体化し，**perform** でも操作型引数を $\alpha := T$ とする．

$$\dfrac{\dfrac{\dfrac{}{z:\theta \vdash z:\theta \mid \varnothing}\,(\text{Abs})}{\vdash \lambda z.z : T \mid \varnothing} \qquad \dfrac{}{id:\sigma_{id} \vdash id : T \mid \varnothing}\,(\text{Perform } \alpha := T)}{\vdash B : T \mid \{ping\}}\,(\text{LetVal})$$

宣言的な Handle 規則の適用． 本体の前提は上で得た．規則全体には return 節と操作節の前提も要る．

$$\dfrac{C;\varnothing \vdash B : T \mid \{ping\} \qquad C; r:T \vdash r : a \mid \varnothing \qquad D; x:\beta,\ k:\beta \xrightarrow{\chi} a \vdash x : \beta \mid \varnothing}{C;\varnothing \vdash H : a \mid \chi}\,(\text{Handle})$$

第 2 前提は $C \models T = a$．操作節では $x:\beta$ が得られ，Handle 規則は $C \models \forall\beta.(D \wedge \beta = a \wedge \varnothing \subseteq \chi)$ を要求する．さらに Split と効果包含が必要である．従って B の型だけでは結論を得られない．

Handler の制約生成． 操作節では別の剛性変数 β を導入する．a は handler の結果型で，$\beta \notin \mathrm{FV}(a)$ である．

$$\dfrac{\mathrm{Gen}_{\varnothing}(B) = (T, \{ping\}, C_B) \qquad \mathrm{Gen}_{r:T}(r) = (T, \varnothing, \top) \qquad \mathrm{Gen}_{x:\beta,\,k:\beta\xrightarrow{\chi}a}(x) = (\beta, \varnothing, \top)}{\mathrm{Gen}_{\varnothing}(H) = (a, \chi, C_H)}\,(\text{Handle})$$

$$C_H = C_B \wedge (T = a) \wedge \underbrace{\forall\beta.(\beta = a \wedge \varnothing \subseteq \chi)}_{\text{操作節の結果型 } \tau_o = a}$$
$$\wedge\, \mathrm{Split}(ping, \{ping\}, \delta) \wedge \delta \subseteq \chi \wedge \varnothing \subseteq \chi.$$

$$\mathrm{Split}(ping, \{ping\}, \delta) \Longrightarrow \delta = \varnothing, \qquad \boxed{\forall\beta.\beta = a \quad \text{は充足不能}}.$$

$\beta = Int$ と $\beta = Bool$ を選ぶと，同じ固定型 a が Int と $Bool$ の両方に等しくなって矛盾する．$C = \top$ では Handle 規則の全称閉包の前提を証明できない．$C = C_H$ とすれば形式上は規則を適用できるが，C_H を満たす評価は存在しない．実際の出力では $\theta = \alpha_{10}$, $T = \alpha_6$, $\beta = \alpha_0$, $a = \alpha_{11}$, $\delta = \mu_7$, $\chi = \mu_8$ に対応する．C_B 内の $(\alpha_{10} \xrightarrow{\varnothing} \alpha_{10}) = \alpha_6$ を解いた形で本体の導出木を表示した．

$ping(x;k) \mapsto k\,x$ なら，$x:\beta$ と $k:\beta \xrightarrow{\chi} a$ から節の結果型は a となり，$\forall\beta.\beta = a$ は生成されない．

4　元の System F^{ξ} で止まる導出木

元の構文には **let** と暗黙の型適用がないため，前頁の式を次の明示的な核の式に直す．

$$\Sigma(\mathit{ping}) = \forall\beta :: \mathbf{T}.\beta \to \beta, \qquad \Delta = \gamma :: \mathbf{T},$$
$$B^{\star} = \mathbf{perform}\ \mathit{ping}[\gamma \xrightarrow{\varnothing} \gamma]\,(\lambda z : \gamma.z),$$
$$H^{\star} = \mathbf{handle}\ B^{\star}\ \mathbf{with}\ \{\mathbf{return}\ r \mapsto r;\ \mathit{ping}(x;k) \mapsto x\}.$$

本体は導出できる．

$$\dfrac{\dfrac{\dfrac{z : \gamma \in (z : \gamma)}{\Delta \mid z : \gamma \vdash z : \gamma \mid \varnothing}\ \textsc{TVar}}{\Delta \mid \varnothing \vdash \lambda z : \gamma.z : \gamma \xrightarrow{\varnothing} \gamma \mid \varnothing}\ \textsc{TAbs}}{\Delta \mid \varnothing \vdash B^{\star} : \gamma \xrightarrow{\varnothing} \gamma \mid \{\mathit{ping}\}}\ \textsc{TPerform}\ \beta := \gamma \xrightarrow{\varnothing} \gamma$$

Handler の導出は操作節で止まる．結果型 $\gamma \xrightarrow{\varnothing} \gamma$，結果効果 $\varnothing$ に具体化すると，THandler の効果条件は $\{\mathit{ping}\} \subseteq \{\mathit{ping}\} \cup \varnothing = \{\mathit{ping}\}$ となり成立する．return 節も TVar から $\Delta \mid r : \gamma \xrightarrow{\varnothing} \gamma \vdash r : \gamma \xrightarrow{\varnothing} \gamma \mid \varnothing$ と導ける．残る前提だけを枠で示す．

$$\Gamma_{op} = x : \beta, \quad k : \beta \xrightarrow{\varnothing} (\gamma \xrightarrow{\varnothing} \gamma).$$

$$\dfrac{\Delta \mid \varnothing \vdash B^{\star} : \gamma \xrightarrow{\varnothing} \gamma \mid \{\mathit{ping}\} \qquad \Delta \mid \varnothing \overset{?}{\vdash} \left\{\begin{array}{l}\mathbf{return}\ r \mapsto r;\\ \mathit{ping}(x;k) \mapsto x\end{array}\right\} : \gamma \xrightarrow{\varnothing} \gamma \mid \{\mathit{ping}\} \Rightarrow \gamma \xrightarrow{\varnothing} \gamma \mid \varnothing}{\Delta \mid \varnothing \overset{?}{\vdash} H^{\star} : \gamma \xrightarrow{\varnothing} \gamma \mid \varnothing}\ \textsc{THandle}$$

$$\dfrac{\Delta \mid r : \gamma \xrightarrow{\varnothing} \gamma \vdash r : \gamma \xrightarrow{\varnothing} \gamma \mid \varnothing \qquad \boxed{\Delta, \beta :: \mathbf{T} \mid \Gamma_{op} \overset{?}{\vdash} x : \gamma \xrightarrow{\varnothing} \gamma \mid \varnothing}}{\Delta \mid \varnothing \overset{?}{\vdash} \left\{\begin{array}{l}\mathbf{return}\ r \mapsto r;\\ \mathit{ping}(x;k) \mapsto x\end{array}\right\} : \gamma \xrightarrow{\varnothing} \gamma \mid \{\mathit{ping}\} \Rightarrow \gamma \xrightarrow{\varnothing} \gamma \mid \varnothing}\ \textsc{THandler}$$

枠は未証明の前提であり，これは完成した導出木ではない．操作節の x に TVar を適用すると，

$$\dfrac{x : \beta \in \Gamma_{op}}{\Delta, \beta :: \mathbf{T} \mid \Gamma_{op} \vdash x : \beta \mid \varnothing}\ \textsc{TVar}$$

x の型は β であって $\gamma \xrightarrow{\varnothing} \gamma$ ではない．TSub で関数型に変えるには

$$\boxed{\Delta, \beta :: \mathbf{T} \vdash \beta <: \gamma \xrightarrow{\varnothing} \gamma}$$

が必要だが，元の部分型規則は型変数に対して反射律 $\beta <: \beta$ しか与えず，外側で固定した関数型には導けない．結果型を別の外側の型 R に変えても $\beta <: R$ が必要になる．効果 $\varnothing \subseteq \epsilon_o$ は障害ではない．したがって $H^{\star}$ には元の System F^{ξ} の完成した型付け導出木がない．前頁の $\forall\beta.\beta = a$ は，同じ障害を EHM(X) の等式制約で表したもの．